



GYMSTATS

Entrega Proyecto



20 DE FEBRERO DE 2026

YON, CARLOS, IAN



Índice

Índice	1
GYMSTATS	2
1 - Diseño perceptivo de requisitos	2
2 - Diseño conceptual	3
3 - Diseño Lógico	3
1 – DDL.....	7
2 – DML	14
3 – Consultas	18
1 – Vistas	28
2 – Funciones y Cursor.....	28
3 – Triggers	32
4 - Interfaz Gráfica y Operaciones CRUD	35



GYMSTATS

1 - Diseño perceptivo de requisitos

Nuestra base de datos está diseñada para la gestión de Gimnasios, con diferentes elementos que pueden tener los mismos, se contemplan varios ámbitos, como los trabajadores, clientes, zonas, clases, etc.

Primero creamos la entidad cliente, en la cual almacenaremos los atributos nombre, apellido y su DNI, los clientes compran membresías, también guardaremos la fecha donde el cliente compra la membresía. Una membresía puede ser comprada por muchos clientes y un cliente puede comprar solo una membresía.

De los clientes con membresía nos interesa tener un registro de entrada y uno de salida, cada uno queremos guardar la fecha y hora, cada registro de acceso guardaremos un código identificativo.

Muchos clientes con diferentes membresías pueden calcular en la calculadora. De la calculadora queremos almacenar el resultado del Id y el IMC.

Muchos clientes con muchas membresías pueden asistir a clases, de las cuales queremos guardar el código y tipo de clase.

Un empleado o varios pueden atender a varios clientes cada vez que se atiende a un cliente se le guarda la descripción de la consulta y la fecha de la misma, para mejorar los servicios del gimnasio.

Hay varios tipos de empleados, de los empleados queremos guardar el DNI, el teléfono, el nombre, el apellido, la nómina, el usuario en la aplicación y la contraseña para logearse. En total hay 4 tipos de empleados: los administradores, de los cuales guardaremos el usuario y contraseña de acceso; los recepcionistas, de los cuales guardaremos el turno; los limpiadores que también guardaremos el turno día/noche y los entrenadores, de los cuales guardaremos el tipo (artes marciales, entrenadores de fuerza, entrenadores de cardio...).

Un entrenador puede enseñar a muchos entrenadores y un entrenador puede ser enseñado por muchos entrenadores.

Una clase en una zona de entreno es realizada por muchos entrenadores, mientras que muchos entrenadores realizan una clase en una zona de entreno...

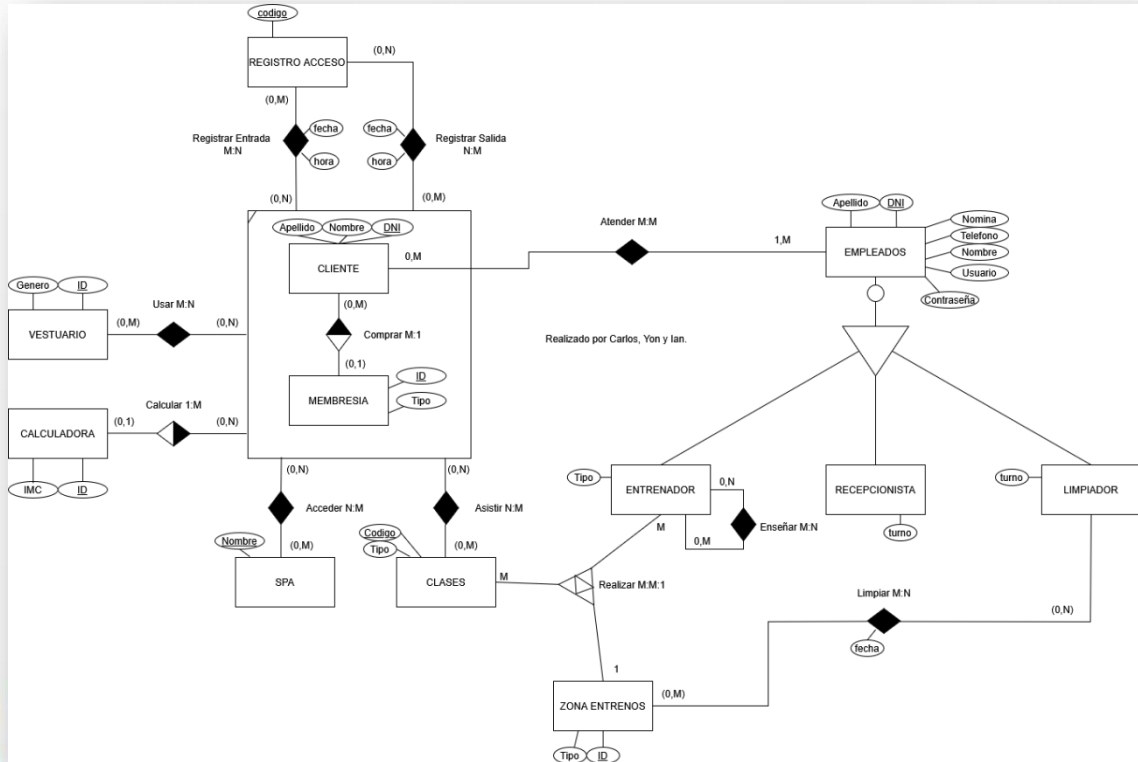
Muchos limpiadores limpian en muchas zonas de entreno, de esta acción almacenamos la fecha.

Por último, contamos con Mensajes de contacto independiente de las otras tablas. En ella almacenamos el identificador, email, teléfono, mensaje y si es una empresa. Su



objetivo es actuar de forma aislada como un buzón de entrada desde la web pública para clientes potenciales y solicitudes externas.

2 - Diseño conceptual



3 - Diseño Lógico

Calculadora (id, IMC)

PK (id)

Registro_Acceso (codigo)

PK (codigo)

Membresia (id, tipo)

PK (id)



Clases (codigo, tipo)

PK (codigo)

Zona Entrenos (id, tipo)

PK (id)

Empleados (dni, telefono, nomina, nombre, apellido, usuario, contraseña)

PK (dni)

Administrador (id, usuario, contraseña, nombre)

PK (id)

Mensajes_Contacto (id, email, telefono, mensaje, es_empresa)

PK (id)

Cliente (dni, nombre, apellido, id_membresia, id_calculadora)

PK (dni)

FK (id_membresia) --> Membresia

FK (id_calculadora) --> Calculadora

Entrenador (tipo, tipo_empleados)

PK (tipo_empleados)

FK (tipo_empleados) --> Empleados



Recepcionista (turno, tipo_empleados)

PK (tipo_empleados)

FK (tipo_empleados) --> Empleados

Limpiador (tipo_empleados, turno)

PK (tipo_empleados)

FK (tipo_empleados) --> Empleados

Registrar_Entrada (dni_cliente, codigo_registro_acceso, fecha, hora)

PK (codigo_registro_acceso, dni_cliente)

FK (codigo_registro_acceso) --> Registro_Acceso

FK (dni_cliente) --> Cliente

Registrar_Salida (dni_cliente, codigo_registro_acceso, fecha, hora)

PK (codigo_registro_acceso, dni_cliente)

FK (codigo_registro_acceso) --> Registro_Acceso

FK (dni_cliente) --> Cliente

Asistir (codigo_clases, dni_cliente)

PK (codigo_clases, dni_cliente)

FK (codigo_clases) --> Clases

FK (dni_cliente) --> Cliente

Atender (dni_empleados, dni_cliente, fecha, descripcion)

PK (dni_empleados, dni_cliente)



FK (dni_cliente) --> Cliente

FK (dni_empleados) --> Empleados

Enseñar (entrenador1, entrenador2)

PK (entrenador1, entrenador2)

FK (entrenador1) --> Entrenador

FK (entrenador2) --> Entrenador

Limpiar (turno_limpiador, id_zona_de_entrenos, fecha)

PK (turno_limpiador, id_zona_de_entrenos)

FK (turno_limpiador) --> Limpiador

FK (id_zona_de_entrenos) --> Zona Entrenos

Entrenar (dni_cliente, id_zona_entrenos)

PK (dni_cliente, id_zona_entrenos)

FK (dni_cliente) --> Cliente

FK (id_zona_entrenos) --> Zona Entrenos

Realizar (dni_entrenador, codigo_clases, id_zona_entrenos, hora, descripcion)

PK (dni_entrenador, codigo_clases)

VNN (id_zona_entrenos)

FK (dni_entrenador) --> Entrenador

FK (codigo_clases) --> Clases

FK (id_zona_entrenos) --> Zona Entreno



1 – DDL

```
DROP TABLE IF EXISTS Mensajes_Contacto CASCADE;
DROP TABLE IF EXISTS Entrenar CASCADE;
DROP TABLE IF EXISTS Limpiar CASCADE;
DROP TABLE IF EXISTS Ensenyar CASCADE;
DROP TABLE IF EXISTS Realizar CASCADE;
DROP TABLE IF EXISTS Atender CASCADE;
DROP TABLE IF EXISTS Asistir CASCADE;
DROP TABLE IF EXISTS Registrar_Salida CASCADE;
DROP TABLE IF EXISTS Registrar_Entrada CASCADE;
DROP TABLE IF EXISTS Cliente CASCADE;
DROP TABLE IF EXISTS Limpiador CASCADE;
DROP TABLE IF EXISTS Recepcionista CASCADE;
DROP TABLE IF EXISTS Entrenador CASCADE;
DROP TABLE IF EXISTS Administrador CASCADE;
DROP TABLE IF EXISTS Empleados CASCADE;
DROP TABLE IF EXISTS Zona_Entrenos CASCADE;
DROP TABLE IF EXISTS Clases CASCADE;
DROP TABLE IF EXISTS Registro_Acceso CASCADE;
DROP TABLE IF EXISTS Calculadora CASCADE;
DROP TABLE IF EXISTS Membresia CASCADE;
```

-- *MEMBRESIA*

```
CREATE TABLE Membresia (
    id INT PRIMARY KEY,
    tipo VARCHAR(50)
);
```

-- *CALCULADORA IMC*

```
CREATE TABLE Calculadora (
```




```
id INT PRIMARY KEY,  
IMC DECIMAL(5,2)  
);
```

-- *REGISTRO DE ACCESO*

```
CREATE TABLE Registro_Acceso (  
codigo INT PRIMARY KEY  
);
```

-- *TIPO DE CLASES*

```
CREATE TABLE Clases (  
codigo INT PRIMARY KEY,  
tipo VARCHAR(50)  
);
```

-- *ZONAS DE ENTRENO*

```
CREATE TABLE Zona_Entrenos (  
id INT PRIMARY KEY,  
tipo VARCHAR(50)  
);
```

-- *DATOS EMPLEADOS*

```
CREATE TABLE Empleados (  
dni VARCHAR(20) PRIMARY KEY,  
telefono VARCHAR(20),  
nomina DECIMAL(10,2),  
nombre VARCHAR(50),  
apellido VARCHAR(50),  
usuario VARCHAR(50) UNIQUE,  
contraseña VARCHAR(50)
```



);

-- *ADMINISTRADORES*

```
CREATE TABLE Administrador (  
    id INT PRIMARY KEY,  
    usuario VARCHAR(50) UNIQUE,  
    contraseña VARCHAR(50),  
    nombre VARCHAR(50)  
);
```

-- *DATOS TIPOS DE EMPLEADOS*

```
CREATE TABLE Entrenador (  
    tipo_empleados VARCHAR(20) PRIMARY KEY,  
    tipo VARCHAR(50),  
    FOREIGN KEY (tipo_empleados) REFERENCES Empleados(dni) ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
CREATE TABLE Recepcionista (  
    tipo_empleados VARCHAR(20) PRIMARY KEY,  
    turno VARCHAR(20),  
    FOREIGN KEY (tipo_empleados) REFERENCES Empleados(dni) ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
CREATE TABLE Limpiador (  
    tipo_empleados VARCHAR(20) PRIMARY KEY,  
    turno VARCHAR(20),  
    FOREIGN KEY (tipo_empleados) REFERENCES Empleados(dni) ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```



-- *DATOS CLIENTES*

```
CREATE TABLE Cliente (  
    dni VARCHAR(20) PRIMARY KEY,  
    nombre VARCHAR(50),  
    apellido VARCHAR(50),  
    usuario VARCHAR(50) UNIQUE,  
    contraseña VARCHAR(50),  
    id_membresia INT,  
    id_calculadora INT,  
    ultimo_acceso TIMESTAMP,  
    fecha_compra DATE DEFAULT CURRENT_DATE,  
    FOREIGN KEY (id_membresia) REFERENCES Membresia(id) ON DELETE SET NULL  
    ON UPDATE CASCADE,  
    FOREIGN KEY (id_calculadora) REFERENCES Calculadora(id) ON DELETE SET NULL  
    ON UPDATE CASCADE  
);
```

-- *REGISTRAR LA ENTRADA/SALIDA*

```
CREATE TABLE Registrar_Entrada (  
    codigo_registro_acceso INT,  
    dni_cliente VARCHAR(20),  
    fecha DATE,  
    hora TIME,  
    PRIMARY KEY (codigo_registro_acceso, dni_cliente),  
    FOREIGN KEY (codigo_registro_acceso) REFERENCES Registro_Acceso(codigo) ON  
    DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (dni_cliente) REFERENCES Cliente(dni) ON DELETE CASCADE ON  
    UPDATE CASCADE  
);
```



```
CREATE TABLE Registrar_Salida (  
    codigo_registro_acceso INT,  
    dni_cliente VARCHAR(20),  
    fecha DATE,  
    hora TIME,  
    PRIMARY KEY (codigo_registro_acceso, dni_cliente),  
    FOREIGN KEY (codigo_registro_acceso) REFERENCES Registro_Acceso(codigo) ON  
DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (dni_cliente) REFERENCES Cliente(dni) ON DELETE CASCADE ON  
UPDATE CASCADE  
);  
  
-- ASISTIR A CLASES  
CREATE TABLE Asistir (  
    codigo_clases INT,  
    dni_cliente VARCHAR(20),  
    PRIMARY KEY (codigo_clases, dni_cliente),  
    FOREIGN KEY (codigo_clases) REFERENCES Clases(codigo) ON DELETE CASCADE  
ON UPDATE CASCADE,  
    FOREIGN KEY (dni_cliente) REFERENCES Cliente(dni) ON DELETE CASCADE ON  
UPDATE CASCADE  
);  
  
-- EMPLEADO ATIENDE A CLIENTE  
CREATE TABLE Atender (  
    dni_empleados VARCHAR(20),  
    dni_cliente VARCHAR(20),  
    fecha DATE,  
    descripcion VARCHAR(255),  
    PRIMARY KEY (dni_empleados, dni_cliente),  
    FOREIGN KEY (dni_empleados) REFERENCES Empleados(dni) ON DELETE CASCADE  
ON UPDATE CASCADE,
```



FOREIGN KEY (dni_cliente) REFERENCES Cliente(dni) ON DELETE CASCADE ON UPDATE CASCADE

);

-- Relación ternaria Realizar -> Entrenadores, Zona Entrenos y Clases

CREATE TABLE Realizar (

dni_entrenador VARCHAR(20),

codigo_clases INT,

id_zona_entrenos INT NOT NULL,

hora TIME,

descripcion VARCHAR(255),

PRIMARY KEY (dni_entrenador, codigo_clases),

FOREIGN KEY (dni_entrenador) REFERENCES Entrenador(tipo_empleados) ON DELETE CASCADE ON UPDATE CASCADE,

FOREIGN KEY (codigo_clases) REFERENCES Clases(codigo) ON DELETE CASCADE ON UPDATE CASCADE,

FOREIGN KEY (id_zona_entrenos) REFERENCES Zona_Entrenos(id) ON DELETE CASCADE ON UPDATE CASCADE

);

-- UN ENTRENADOR PUEDE ENSEÑAR A OTRO ENTRENADOR

CREATE TABLE Ensenyar (

entrenador1 VARCHAR(20),

entrenador2 VARCHAR(20),

PRIMARY KEY (entrenador1, entrenador2),

FOREIGN KEY (entrenador1) REFERENCES Entrenador(tipo_empleados) ON DELETE CASCADE ON UPDATE CASCADE,

FOREIGN KEY (entrenador2) REFERENCES Entrenador(tipo_empleados) ON DELETE CASCADE ON UPDATE CASCADE

);

-- LIMPIADORES LIMPIAN ZONAS



```
CREATE TABLE Limpiar (  
    turno_limpiador VARCHAR(20),  
    id_zona_de_entrenos INT,  
    fecha DATE,  
    PRIMARY KEY (turno_limpiador, id_zona_de_entrenos),  
    FOREIGN KEY (turno_limpiador) REFERENCES Limpiador(tipo_empleados) ON  
DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (id_zona_de_entrenos) REFERENCES Zona_Entrenos(id) ON DELETE  
CASCADE ON UPDATE CASCADE  
);
```

-- *ENTRENAR Clientes -> zonas de entreno*

```
CREATE TABLE Entrenar (  
    dni_cliente VARCHAR(20),  
    id_zona_entrenos INT,  
    PRIMARY KEY (dni_cliente, id_zona_entrenos),  
    FOREIGN KEY (dni_cliente) REFERENCES Cliente(dni) ON DELETE CASCADE ON  
UPDATE CASCADE,  
    FOREIGN KEY (id_zona_entrenos) REFERENCES Zona_Entrenos(id) ON DELETE  
CASCADE ON UPDATE CASCADE  
);
```

-- *Contacto que llega de la página web*

```
CREATE TABLE Mensajes_Contacto (  
    id SERIAL PRIMARY KEY,  
    email VARCHAR(100),  
    telefono VARCHAR(20),  
    mensaje TEXT,  
    es_empresa BOOLEAN  
);
```



2 – DML

INSERT INTO Membresia (id, tipo) VALUES

(1, 'Básica'),
(2, 'Premium'),
(3, 'Familiar'),
(4, 'Estudiante'),
(5, 'VIP');

INSERT INTO Calculadora (id, IMC) VALUES

(1, 22.50),
(2, 27.80),
(3, 19.40),
(4, 24.10),
(5, 30.25);

INSERT INTO Registro_Acceso (codigo) VALUES

(101),
(102),
(103),
(104),
(105);

INSERT INTO Clases (codigo, tipo) VALUES

(501, 'Yoga'),
(502, 'Crossfit'),
(503, 'Pilates'),
(504, 'Zumba'),
(505, 'Boxeo');



INSERT INTO Zona_Entrenos (id, tipo) VALUES

(1, 'Cardio'),
(2, 'Musculación'),
(3, 'Funcional'),
(4, 'Peso Libre'),
(5, 'Estiramientos');

INSERT INTO Empleados (dni, telefono, nomina, nombre, apellido, usuario, contraseña) VALUES

('11111111A', '600111222', 1500.00, 'Carlos', 'Sánchez', 'carlos_emp', '1234'),
('22222222B', '600333444', 1200.00, 'Laura', 'Gómez', 'laura', '1234'),
('33333333C', '600555666', 1100.00, 'Pedro', 'Ruiz', 'pedro', '1234'),
('44444444D', '600777888', 1800.00, 'Ana', 'Martínez', 'ana', '1234'),
('55555555E', '600999000', 1600.00, 'Sergio', 'Fernández', 'sergio', '1234');

INSERT INTO Entrenador (tipo_empleados, tipo) VALUES

('11111111A', 'Musculación'),
('44444444D', 'Cardio'),
('55555555E', 'Artes Marciales'),
('22222222B', 'Yoga'),
('33333333C', 'Funcional');

INSERT INTO Recepcionista (tipo_empleados, turno) VALUES

('22222222B', 'Mañana'),
('11111111A', 'Tarde'),
('33333333C', 'Noche');



```
INSERT INTO Limpiador (tipo_empleados, turno) VALUES
```

```
('33333333C', 'Tarde'),
```

```
('11111111A', 'Mañana');
```

```
INSERT INTO Cliente (dni, nombre, apellido, usuario, contraseña, id_membresia,  
id_calculadora, ultimo_acceso, fecha_compra) VALUES
```

```
('99999999Z', 'Juan', 'Pérez', 'juanp', '1234', 2, 1, '2023-10-25 09:00:00', '2023-01-15'),
```

```
('88888888Y', 'Maria', 'López', 'marial', '1234', 1, 2, '2023-10-25 10:30:00', '2023-03-10'),
```

```
('77777777X', 'Luis', 'Torres', 'luist', '1234', 3, 3, '2023-10-26 08:45:00', '2023-06-20'),
```

```
('66666666W', 'Sara', 'Díaz', 'sarad', '1234', 1, 1, '2023-10-26 11:00:00', '2023-09-05'),
```

```
('55555555V', 'Marta', 'Ramos', 'martar', '1234', 2, 2, '2023-10-27 09:30:00', '2023-10-01');
```

```
INSERT INTO Registrar_Entrada (codigo_registro_acceso, dni_cliente, fecha, hora)  
VALUES
```

```
(101, '99999999Z', '2023-10-25', '09:00:00'),
```

```
(101, '88888888Y', '2023-10-25', '10:30:00'),
```

```
(102, '77777777X', '2023-10-26', '08:45:00'),
```

```
(102, '66666666W', '2023-10-26', '11:00:00'),
```

```
(103, '55555555V', '2023-10-27', '09:30:00');
```

```
INSERT INTO Registrar_Salida (codigo_registro_acceso, dni_cliente, fecha, hora)  
VALUES
```

```
(101, '99999999Z', '2023-10-25', '11:00:00'),
```

```
(101, '88888888Y', '2023-10-25', '12:00:00'),
```

```
(102, '77777777X', '2023-10-26', '10:30:00'),
```

```
(102, '66666666W', '2023-10-26', '12:30:00'),
```

```
(103, '55555555V', '2023-10-27', '11:00:00');
```



```
INSERT INTO Asistir (codigo_clases, dni_cliente) VALUES
```

```
(501, '88888888Y'),
```

```
(502, '99999999Z'),
```

```
(503, '77777777X'),
```

```
(501, '66666666W');
```

```
INSERT INTO Atender (dni_empleados, dni_cliente, fecha, descripcion) VALUES
```

```
('22222222B', '99999999Z', '2023-10-25', 'Consulta sobre horarios y membresía'),
```

```
('22222222B', '88888888Y', '2023-10-25', 'Alta de nuevo cliente'),
```

```
('11111111A', '77777777X', '2023-10-26', 'Revisión de rutina de entrenamiento'),
```

```
('44444444D', '55555555V', '2023-10-26', 'Evaluación física inicial');
```

```
INSERT INTO Ensenyar (entrenador1, entrenador2) VALUES
```

```
('44444444D', '11111111A'),
```

```
('55555555E', '44444444D');
```

```
INSERT INTO Limpiar (turno_limpiador, id_zona_de_entrenos, fecha) VALUES
```

```
('33333333C', 1, '2023-10-25'),
```

```
('33333333C', 2, '2023-10-25'),
```

```
('33333333C', 3, '2023-10-26');
```

```
INSERT INTO Realizar (dni_entrenador, codigo_clases, id_zona_entrenos, hora, descripcion) VALUES
```

```
('11111111A', 502, 2, '10:00:00', 'Clase intensa de levantamiento'),
```

```
('44444444D', 501, 3, '18:00:00', 'Yoga para principiantes'),
```

```
('55555555E', 503, 1, '19:30:00', 'Pilates avanzado'),
```

```
('22222222B', 504, 4, '09:00:00', 'Sesión de Zumba cardio'),
```

```
('33333333C', 505, 5, '20:00:00', 'Entrenamiento de Boxeo y técnica');
```



```
INSERT INTO Entrenar (dni_cliente, id_zona_entrenos) VALUES
```

```
('99999999Z', 2),
```

```
('88888888Y', 1),
```

```
('77777777X', 3),
```

```
('66666666W', 1),
```

```
('55555555V', 2);
```

```
INSERT INTO Administrador (id, usuario, contraseña, nombre) VALUES
```

```
(1, 'carlos', '1234', 'Carlos'),
```

```
(2, 'admin', 'admin', 'Administrador Principal');
```

```
INSERT INTO Mensajes_Contacto (email, telefono, mensaje, es_empresa) VALUES
```

```
('juan.interesado@email.com', '611222333', 'Hola, ¿tenéis entrenadores personales disponibles por la tarde?', FALSE),
```

```
('marta.gomez@email.com', '622333444', 'Me gustaría probar un día las instalaciones, ¿es posible?', FALSE),
```

```
('info@suplementosgym.es', '600123456', 'Buenas, somos una marca de suplementos y nos gustaría patrocinarnos.', TRUE),
```

```
('lucas.fitness@email.com', '633444555', 'Quiero apuntarme al gimnasio el mes que viene, ¿hay matrícula?', FALSE);
```

3 – Consultas

-- a. 5 Consultas simples de una sola tabla

-- Todos los empleados

```
SELECT *
```

```
FROM Empleados;
```



	dni [PK] character varying (20)	telefono character varying (20)	nomina numeric (10,2)	nombre character varying (50)	apellido character varying (50)
1	11111111A	600111222	1500.00	Carlos	Sánchez
2	22222222B	600333444	1200.00	Laura	Gómez
3	33333333C	600555666	1100.00	Pedro	Ruiz
4	44444444D	600777888	1800.00	Ana	Martínez
5	55555555E	600999000	1600.00	Sergio	Fernández

-- Nombres y apellidos de clientes

SELECT nombre, apellido

FROM Cliente;

	nombre character varying (50)	apellido character varying (50)
1	Juan	Pérez
2	Maria	López
3	Luis	Torres
4	Sara	Díaz
5	Marta	Ramos

-- Tipos de membresias

SELECT tipo

FROM Membresia;

	tipo character varying (50)
1	Básica
2	Premium
3	Familiar
4	Estudiante
5	VIP

-- Tipos de zonas de entreno

SELECT tipo

FROM Zona_Entrenos;



	tipo character varying (50)
1	Cardio
2	Musculación
3	Funcional
4	Peso Libre
5	Estiramientos

-- Correos del formulario de contacto web

```
SELECT email
```

```
FROM Mensajes_Contacto;
```

	email character varying (100)
1	juan.interesado@email.c...
2	marta.gomez@email.com
3	info@suplementosgym.es
4	lucas.fitness@email.com

-- b. Actualizaciones y 2 Borrados en cualquier tabla

-- Actualizar telefono de un empleado

```
UPDATE Empleados
```

```
SET telefono = '699000000'
```

```
WHERE dni = '11111111A';
```

-- Actualizar nombre de una clase

```
UPDATE Clases
```

```
SET tipo = 'Yoga Avanzado'
```

```
WHERE codigo = 501;
```

-- Borrar un acceso



DELETE

FROM Registro_Acceso

WHERE codigo = 105;

-- Borrar un mensaje de contacto de la web

DELETE

FROM Mensajes_Contacto

WHERE id = 1;

-- c. 3 Consultas con más de 1 tabla

-- Cliente y su membresia

SELECT Cliente.nombre, Membresia.tipo

FROM Cliente, Membresia

WHERE Cliente.id_membresia = Membresia.id;

	nombre character varying (50) 🔒	tipo character varying (50) 🔒
1	Juan	Premium
2	Maria	Básica
3	Luis	Familiar
4	Sara	Básica
5	Marta	Premium

-- Entrenador y el codigo de sus clases

SELECT Empleados.nombre, Realizar.codigo_clases

FROM Empleados, Realizar

WHERE Empleados.dni = Realizar.dni_entrenador;

	nombre character varying (50) 🔒	codigo_clases integer 🔒
1	Carlos	502
2	Ana	501
3	Sergio	503
4	Laura	504
5	Pedro	505

-- Cliente y su IMC

SELECT Cliente.nombre, Calculadora.IMC

FROM Cliente, Calculadora



WHERE Cliente.id_calculadora = Calculadora.id;

	nombre character varying (50) 🔒	imc numeric (5,2) 🔒
1	Juan	22.50
2	Maria	27.80
3	Luis	19.40
4	Sara	22.50
5	Marta	27.80

-- d. 3 Consultas usando funciones

-- Total gasto en nominas

SELECT SUM(nomina)
FROM Empleados;

	sum numeric 🔒
1	7200.00

-- Numero total de clientes

SELECT COUNT(*)
FROM Cliente;

	count bigint 🔒
1	5

-- IMC mas alto

SELECT MAX(IMC)
FROM Calculadora;

	max numeric 🔒
1	30.25

-- e. 2 Consultas usando group by

-- Clientes por id de membresia

SELECT id_membresia, COUNT(*)
FROM Cliente
GROUP BY id_membresia;

	id_membresia integer 🔒	count bigint 🔒
1	3	1
2	2	2
3	1	2



-- Recepcionistas por turno

```
SELECT turno, COUNT(*)
FROM Recepcionista
GROUP BY turno;
```

	turno character varying (20)	count bigint
1	Tarde	1
2	Mañana	1
3	Noche	1

-- f. 2 Consultas utilizando subconsultas

-- Clientes con IMC mayor a la media

```
SELECT nombre FROM Cliente
WHERE id_calculadora IN (
    SELECT id
    FROM Calculadora
    WHERE IMC > (
        SELECT AVG(IMC)
        FROM Calculadora)
);
```

	nombre character varying (50)
1	Maria
2	Marta

-- Empleados que cobran mas que Carlos Sánchez

```
SELECT nombre, apellido FROM Empleados
WHERE nomina > (
    SELECT nomina
    FROM Empleados
    WHERE nombre = 'Carlos'
    AND apellido = 'Sánchez'
);
```

	nombre character varying (50)	apellido character varying (50)
1	Ana	Martínez
2	Sergio	Fernández

-- g. 2 Consultas usando group by con having

**-- Membresias con un cliente o mas de 1 clientes**

```
SELECT id_membresia, COUNT(*)  
FROM Cliente  
GROUP BY id_membresia  
HAVING COUNT(*) >= 1;
```

	id_membresia integer	count bigint
1	3	1
2	2	3
3	1	1

-- Turnos de limpieza con empleados

```
SELECT turno, COUNT(*)  
FROM Limpiador  
GROUP BY turno  
HAVING COUNT(*) > 0;
```

	turno character varying (20)	count bigint
1	Tarde	1
2	Mañana	1

-- h. 3 actualizaciones usando subconsultas en where y set**-- Subir sueldo a entrenadores de musculacion**

```
UPDATE Empleados  
SET nomina = nomina * 1.10  
WHERE dni IN (  
    SELECT tipo_empleados  
    FROM Entrenador  
    WHERE tipo = 'Musculatura'  
);
```

-- Cambiar a membresia 2 a clientes con IMC mayor a 27

```
UPDATE Cliente  
SET id_membresia = 2  
WHERE id_calculadora IN (  
    SELECT id  
    FROM Calculadora  
    WHERE IMC > 27  
);
```

**-- Actualizar descripcion de atencion de Laura Gomez**

UPDATE Atender

SET descripcion = 'Seguimiento prioritario'

WHERE dni_empleados = (

SELECT dni

FROM Empleados

WHERE nombre = 'Laura'

AND apellido = 'Gómez'

);



-- EXTRAS

-- Datos completos del cliente

```
SELECT C.nombre, C.apellido, M.tipo, Ca.IMC
FROM Cliente C, Membresia M, Calculadora Ca
WHERE C.id_membresia = M.id
AND C.id_calculadora = Ca.id;
```

	nombre character varying (50) 🔒	apellido character varying (50) 🔒	tipo character varying (50) 🔒	imc numeric (5,2) 🔒
1	Juan	Pérez	Premium	22.50
2	Luis	Torres	Familiar	19.40
3	Sara	Díaz	Básica	22.50
4	Maria	López	Premium	27.80
5	Marta	Ramos	Premium	27.80

-- Entrenador, clase y zona de entreno

```
SELECT E.nombre, Cl.tipo, Z.tipo
FROM Realizar R, Empleados E, Clases Cl, Zona_Entrenos Z
WHERE R.dni_entrenador = E.dni
AND R.codigo_clases = Cl.codigo
AND R.id_zona_entrenos = Z.id;
```

	nombre character varying (50) 🔒	tipo character varying (50) 🔒	tipo character varying (50) 🔒
1	Carlos	Crossfit	Musculación
2	Ana	Yoga Avanzado	Funcional
3	Sergio	Pilates	Cardio
4	Laura	Zumba	Peso Libre
5	Pedro	Boxeo	Estiramientos



-- Clientes y la fecha de sus registros de entrada

```
SELECT C.nombre, R.fecha
FROM Cliente C, Registrar_Entrada R
WHERE C.dni = R.dni_cliente;
```

	nombre character varying (50) 🔒	fecha date 🔒
1	Juan	2023-10-...
2	Maria	2023-10-...
3	Luis	2023-10-...
4	Sara	2023-10-...
5	Marta	2023-10-...

-- Entrenadores que imparten más clases o las mismas que 'Carlos Sánchez'

```
SELECT E.nombre, E.apellido, COUNT(R.codigo_clases) AS Total_Clases
FROM Empleados E, Entrenador En, Realizar R
WHERE E.dni = En.tipo_empleados
      AND En.tipo_empleados = R.dni_entrenador
GROUP BY E.nombre, E.apellido
HAVING COUNT(R.codigo_clases) >= (
    SELECT COUNT(R2.codigo_clases)
    FROM Empleados E2, Realizar R2
    WHERE E2.dni = R2.dni_entrenador
          AND E2.nombre = 'Carlos'
          AND E2.apellido = 'Sánchez'
);
```

	nombre character varying (50) 🔒	apellido character varying (50) 🔒	total_clases bigint 🔒
1	Laura	Gómez	1
2	Sergio	Fernández	1
3	Carlos	Sánchez	1
4	Pedro	Ruiz	1
5	Ana	Martínez	1



1 – Vistas

-- VISTAS

-- **Vista 1: Clientes con su membresia**

-- Une la tabla Cliente con Membresia para ver de un vistazo

-- que tipo de membresia tiene cada socio del gimnasio.

```
CREATE OR REPLACE VIEW vista_clientes_membresia AS
SELECT c.dni, c.nombre, c.apellido, m.tipo AS tipo_membresia
FROM Cliente c, Membresia m
WHERE c.id_membresia = m.id;
```

-- **Vista 2: Clases con el entrenador que las imparte**

-- Une Clases, Realizar y Empleados para ver que entrenador

-- da cada clase y a que hora.

```
CREATE OR REPLACE VIEW vista_clases_entrenador AS
SELECT cl.codigo, cl.tipo AS tipo_clase, r.hora, e.nombre AS nombre_entrenador,
e.apellido AS apellido_entrenador
FROM Clases cl, Realizar r, Empleados e
WHERE cl.codigo = r.codigo_clases
AND r.dni_entrenador = e.dni;
```

2 – Funciones y Cursor

-- **Funcion 1: Contar clientes según su tipo de membresía**

-- Le pasas el tipo ('Basica', 'Premium'...) y te dice cuántos clientes la tienen.

```
CREATE OR REPLACE FUNCTION contar_clientes_por_membresia(p_tipo
VARCHAR)
RETURNS INT AS $$
DECLARE
    total INT;
```



BEGIN

SELECT COUNT(*)

INTO total

FROM Cliente c, Membresia m

WHERE c.id_membresia = m.id

AND m.tipo = p_tipo;

RETURN total;

END;

\$\$ LANGUAGE plpgsql;

-- Ejemplo de uso:

-- SELECT contar_clientes_por_membresia('Premium');

-- **Funcion 2: Ver la nomina de un empleado**

-- Le pasas el DNI y te devuelve su nomina.

-- Si el empleado no existe devuelve -1.

CREATE OR REPLACE FUNCTION obtener_nomina_empleado(p_dni VARCHAR)

RETURNS DECIMAL(10,2) AS \$\$

DECLARE

v_nomina DECIMAL(10,2);

BEGIN

SELECT nomina

INTO v_nomina

FROM Empleados

WHERE dni = p_dni;

IF NOT FOUND THEN



```
        RETURN -1;

    END IF;

    RETURN v_nomina;

END;

$$ LANGUAGE plpgsql;

-- Ejemplo de uso:
-- SELECT obtener_nomina_empleado('12345678A');

-- Funcion 3: Ver cuantas clases tiene un entrenador
-- Le pasas el DNI del entrenador y te dice cuantas clases imparte.
CREATE OR REPLACE FUNCTION clases_por_entrenador(p_dni VARCHAR)
RETURNS INT AS $$
DECLARE
    total INT;
BEGIN
    SELECT COUNT(*)
    INTO total
    FROM Realizar
    WHERE dni_entrenador = p_dni;

    RETURN total;
END;

$$ LANGUAGE plpgsql;

-- Ejemplo de uso:
-- SELECT clases_por_entrenador('12345678A');
```



-- Funcion 4 (CON CURSOR): Listar clientes con su membresia

-- Usa un cursor para recorrer todos los clientes uno a uno

-- y devuelve su nombre completo junto al tipo de membresia.

-- Si no tiene membresia asignada aparece 'Sin membresia'.

```
CREATE OR REPLACE FUNCTION listar_clientes_con_membresia()
RETURNS TABLE(nombre_completo TEXT, tipo_membresia TEXT) AS $$
DECLARE
    cur_clientes CURSOR FOR
        SELECT c.nombre, c.apellido, m.tipo
        FROM Cliente c, Membresia m
        WHERE c.id_membresia = m.id;

    v_nombre VARCHAR(50);
    v_apellido VARCHAR(50);
    v_tipo VARCHAR(50);
BEGIN
    OPEN cur_clientes;

    LOOP
        FETCH cur_clientes INTO v_nombre, v_apellido, v_tipo;
        EXIT WHEN NOT FOUND;

        nombre_completo := v_nombre || ' ' || v_apellido;
        tipo_membresia := COALESCE(v_tipo, 'Sin membresia');
        RETURN NEXT;
    END LOOP;
```




```
CLOSE cur_clientes;  
END;  
$$ LANGUAGE plpgsql;
```

```
-- Ejemplo de uso:  
-- SELECT * FROM listar_clientes_con_membresia();
```

3 – Triggers

-- Trigger 1 (GLOBAL - FOR EACH STATEMENT)

```
-- No se puede borrar ningun administrador del sistema.  
-- Al ser de tipo STATEMENT se ejecuta una vez por sentencia DELETE  
-- y la bloquea entera, sin importar cuantas filas afecte.
```

```
CREATE OR REPLACE FUNCTION fn_no_borrar_administradores()  
RETURNS TRIGGER AS $$  
BEGIN  
    RAISE EXCEPTION 'No se permite eliminar administradores del sistema.';  
END;  
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_no_borrar_administradores ON Administrador;  
CREATE TRIGGER trg_no_borrar_administradores  
BEFORE DELETE ON Administrador  
FOR EACH STATEMENT  
EXECUTE FUNCTION fn_no_borrar_administradores();
```

-- Trigger 2 (GLOBAL - FOR EACH STATEMENT)

```
-- No se puede borrar ningun cliente del sistema.  
-- Igual que el anterior, actua a nivel de sentencia y
```



-- cancela cualquier DELETE sobre la tabla Cliente.

```
CREATE OR REPLACE FUNCTION fn_no_borrar_clientes()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    RAISE EXCEPTION 'No se permite eliminar clientes desde este sistema.';
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_no_borrar_clientes ON Cliente;
```

```
CREATE TRIGGER trg_no_borrar_clientes
```

```
BEFORE DELETE ON Cliente
```

```
FOR EACH STATEMENT
```

```
EXECUTE FUNCTION fn_no_borrar_clientes();
```

-- **Trigger 3 (POR FILA - FOR EACH ROW)**

-- Cada vez que un cliente registra una entrada al gimnasio,

-- se actualiza automaticamente su campo ultimo_acceso en la tabla Cliente.

-- Asi no hay que hacerlo a mano cada vez que alguien entre.

```
CREATE OR REPLACE FUNCTION fn_actualizar_ultimo_acceso()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    UPDATE Cliente
```

```
    SET ultimo_acceso = NOW()
```

```
    WHERE dni = NEW.dni_cliente;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```



```
DROP TRIGGER IF EXISTS trg_actualizar_ultimo_acceso ON Registrar_Entrada;
CREATE TRIGGER trg_actualizar_ultimo_acceso
AFTER INSERT ON Registrar_Entrada
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_ultimo_acceso();
```

-- Trigger 4 (POR FILA - FOR EACH ROW)

-- Antes de insertar o modificar un empleado, comprueba que
-- la nomina no sea un numero negativo. Si lo es, lanza un error
-- y no deja guardar el dato.

```
CREATE OR REPLACE FUNCTION fn_validar_nomina()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.nomina IS NOT NULL AND NEW.nomina < 0 THEN
        RAISE EXCEPTION 'La nomina no puede ser negativa. Valor introducido: %',
NEW.nomina;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_validar_nomina ON Empleados;
CREATE TRIGGER trg_validar_nomina
BEFORE INSERT OR UPDATE ON Empleados
FOR EACH ROW
EXECUTE FUNCTION fn_validar_nomina();
```



4 - Interfaz Gráfica y Operaciones CRUD



Para gestionar la base de datos hemos creado una aplicación de escritorio con Java Swing conectada a PostgreSQL mediante JDBC. La interfaz está diseñada para realizar las cuatro operaciones básicas CRUD (Crear, Leer, Actualizar y Borrar) de forma sencilla y directa. Para la lectura de los datos, las ventanas principales utilizan consultas SELECT y muestran la información en tiempo real a través de tablas visuales. Para la creación, hemos programado ventanas emergentes donde el usuario rellena los datos y el sistema ejecuta una sentencia INSERT.



NUEVO SOCIO

DNI del Socio

Nombre

Apellido

Usuario (Login)

Contraseña

GUARDAR SOCIO

Cancelar

La misma situación sucede con modificar información mediante comandos UPDATE

MODIFICAR SOCIO

DNI del Socio (No modificable)

88888888Y

Nombre

Maria

Apellido

López

Usuario (Login)

marial

Contraseña

....

Días de Acceso Restantes

0

ACTUALIZAR DATOS

Cancelar

o eliminarlo por completo con un DELETE, lo cual activa de forma segura el borrado en cascada que configuramos en la base de datos.



Añadir

Modificar

Eliminar

Actualizar

DNI	Nombre	Apellido	Membresía	
88888888Y	Maria	López	1	27.80
77777777X	Luis	Torres	3	19.40
66666666W	Sara	Díaz	1	22.50
55555555V				27.80
99999999Z				22.50

Eliminar

×

?

¿Seguro que quieres eliminar a este usuario y todo su historial?

Sí

No

Cancelar